Lahore Urban Estates Analytics Warehouse

Grain Definition

DL3004 Data Warehousing and Business Intelligence — Final Project — Phase 1 · Version 2.1
FAST-NU Lahore · Fall 2026
Course Instructor: Mr. Muhammad Ishaq Raza · Lab Instructor: Mr. Shahzeb Mubashar
Prepared by: Junaid Minhas (23L-2559), Ahmad Waqar (23L-2634), Hanan Tiwana (23L-2536) and Eesha Khurram (23L-2641)
Date: 16 October 2026

The grain is the contract of a fact table: one sentence stating what a single row represents, precise enough that two developers would build the same table (Lab 04 Task 3.1, Lab 05 Section 4). The grains were fixed before any measures or dimensions were chosen, and each sentence below is used word for word in the diagrams and the data dictionary.

1. Summary of Fact Grains

fact_rent_invoice_line — *Transaction*

> ***One row represents one charge line (rent, service charge, late fee, utility or credit note) on one invoice issued to one lease.***

fact_payment — *Transaction*

> ***One row represents one portion of one payment receipt allocated to one invoice line.***

fact_unit_occupancy_daily — *Periodic snapshot*

> ***One row represents one unit at the end of one calendar day, whether occupied or vacant.***

fact_maintenance_request — *Accumulating snapshot*

> ***One row represents one maintenance request across its whole lifecycle, updated as it is assigned, resolved and verified.***

fact_lease_event — *Factless (event tracking)*

> ***One row represents one lifecycle event (signed, renewed, notice given or terminated) on one lease.***

fact_vacancy_incentive_coverage — *Factless (coverage)*

> ***One row represents one incentive offer being available on one vacant unit on one calendar day.***

2. Source Grains (Lab 05 Task 3.1)

Source grains were profiled first. Invoice headers and invoice lines have different grains, just like order headers and order lines in Lab 05, so they are never loaded into one fact.

- **src_pms.buildings** (S1): One building. *Natural key: building_code.*
- **src_pms.units** (S1): The current state of one unit. *Natural key: unit_id (business key building_code + unit_no).*

- **src_pms.tenants** (S1): The current state of one tenant (no history kept). *Natural key: tenant_code.*
- **src_pms.leases** (S1): One lease contract for one unit and one primary tenant. *Natural key: lease_number.*
- **src_pms.lease_events** (S1): One lifecycle event on one lease. *Natural key: event_id.*
- **src_pms.invoices** (S1): One invoice header issued to one lease for one month. *Natural key: invoice_number.*
- **src_pms.invoice_lines** (S1): One charge line on one invoice. *Natural key: invoice_number + line_no.*
- **src_pms.charge_types** (S1): One charge type. *Natural key: charge_code.*
- **src_pms.payments** (S1): One payment receipt from a tenant. *Natural key: payment_reference.*
- **src_pms.payment_allocations** (S1): One portion of one payment applied to one invoice line. *Natural key: payment_reference + invoice_number + line_no.*
- **src_pms.payment_methods** (S1): One payment method. *Natural key: method_code.*
- **owners (sheet)** (S2): One owner. *Natural key: owner_code.*
- **ownership (sheet)** (S2): One period during which one owner held one unit. *Natural key: unit + owner + valid_from.*
- **vendors (sheet)** (S2): The current state of one vendor. *Natural key: vendor_code.*
- **sla_policy (sheet)** (S2): One SLA rule for one subcategory from a start date. *Natural key: subcategory_code + valid_from.*
- **holidays (sheet)** (S2): One public holiday or Ramadan day. *Natural key: date.*
- **settlement_<channel>_<date>.csv** (S3): One settled transaction on one channel on one day. *Natural key: channel + settlement_id.*
- **tickets.json** (S4): One maintenance request with an array of status changes. *Natural key: ticket_id.*
- **Open-Meteo daily response** (S5): One city on one day. *Natural key: city + date.*
- **incentive_log.csv** (S6): One offer on one unit between a start and an end date. *Natural key: building + unit + offer + start.*

3. Grain Rules Applied

- Facts of different grains are never mixed in one table. Invoicing and payment are two facts because one invoice line can be paid in several instalments and one payment can cover several lines.
- Every dimension attached to a fact is true at that fact's grain, for example the tenant version valid on the paid date.
- Measures are classified by additivity before any aggregation is written (Lab 05 Section 6).
- A grain check query is defined for every fact and runs after each load.

4. fact_rent_invoice_line

Grain (Transaction):

> ***One row represents one charge line (rent, service charge, late fee, utility or credit note) on one invoice issued to one lease.***

Business process: P1 Rent invoicing and payment
Primary key: invoice_line_sk
Load pattern: Incremental insert by invoice date watermark; credit notes arrive as new negative lines
Expected rows (3 years): ≈ 130,000
This grain answers: What was billed, by charge type, lease, unit, owner and due month; credit notes.
This grain cannot answer: When or how it was paid (needs fact_payment).

Dimensions at this grain

- invoice_date_key → dim_date
- due_date_key → dim_date
- unit_key → dim_unit
- tenant_key → dim_tenant
- lease_key → dim_lease
- owner_key → dim_owner
- charge_type_key → dim_charge_type

Measures and additivity

- **billed_amount, tax_amount, discount_amount, net_amount** — Additive. SUM across every dimension.

Example row

```
invoice_line_sk             88123
invoice_date_key            20260301
due_date_key                20260310
unit_key                    117
tenant_key                  402
lease_key                   2210
owner_key                   57
charge_type_key             1
invoice_number              INV-2026-03-00831
invoice_line_no             1
billed_amount               85000.00
tax_amount                  0.00
discount_amount             0.00
net_amount                  85000.00
```

Grain check, run after every load

```
SELECT invoice_number, invoice_line_no, COUNT(*)
FROM lue_dw.fact_rent_invoice_line
GROUP BY invoice_number, invoice_line_no
HAVING COUNT(*) > 1;          -- must return no rows
```

5. fact_payment

Grain (Transaction):

> ***One row represents one portion of one payment receipt allocated to one invoice line.***

Business process: P1 Rent invoicing and payment
Primary key: payment_alloc_sk
Load pattern: Incremental insert by paid date watermark; a bounced cheque adds a new negative row
Expected rows (3 years): ≈ 60,000
This grain answers: When and how money arrived, lateness per line, reversals, settlement lag.
This grain cannot answer: Amounts billed but never paid (needs fact_rent_invoice_line).

Dimensions at this grain

- paid_date_key → dim_date
- due_date_key → dim_date
- unit_key → dim_unit
- tenant_key → dim_tenant
- lease_key → dim_lease
- owner_key → dim_owner
- payment_method_key → dim_payment_method
- charge_type_key → dim_charge_type

Measures and additivity

- **payment_amount** — Additive (signed). SUM; reversals net out.
- **days_past_due, settlement_lag_days** — Non-additive. Amount-weighted average or median.

Example row

```
payment_alloc_sk            40411
paid_date_key               20260314
due_date_key                20260310
unit_key                    117
tenant_key                  402
lease_key                   2210
owner_key                   57
payment_method_key          5
charge_type_key             1
payment_reference           PAY-0098812
invoice_number              INV-2026-03-00831
invoice_line_no             1
payment_amount              50000.00
days_past_due               4
settlement_lag_days         1
is_reversal                 FALSE
```

Grain check, run after every load

```
SELECT payment_reference, invoice_number, invoice_line_no, is_reversal, COUNT(*)
FROM lue_dw.fact_payment
GROUP BY payment_reference, invoice_number, invoice_line_no, is_reversal
HAVING COUNT(*) > 1;          -- must return no rows
```

6. fact_unit_occupancy_daily

Grain (Periodic snapshot):

> ***One row represents one unit at the end of one calendar day, whether occupied or vacant.***
>
> **Business process:** P2 Unit occupancy
> **Primary key:** (snapshot_date_key, unit_key)
> **Load pattern:** Derived by the ETL from leases × dim_date; the last 35 days are re-derived on every run
> **Expected rows (3 years):** ≈ 1.97 million
> **This grain answers:** The state of every unit on every day: occupancy, rent roll, current arrears, vacancy spells.
> **This grain cannot answer:** Why a unit became vacant, or a departed tenant's debt.

Dimensions at this grain

- snapshot_date_key → dim_date
- unit_key → dim_unit
- tenant_key → dim_tenant
- lease_key → dim_lease
- owner_key → dim_owner

Measures and additivity

- **is_occupied** — Semi-additive. SUM across units on one day; AVG across days.
- **contract_rent** — Semi-additive. SUM across units on one day = rent roll; never SUM across days.
- **current_lease_arrears** — Semi-additive. Period-end value; never SUM across days.

Example row

```
snapshot_date_key           20260305
unit_key                    117
tenant_key                  402
lease_key                   2210
owner_key                   57
is_occupied                 1
contract_rent               85000.00
current_lease_arrears       35000.00
```

Grain check, run after every load

```
SELECT COUNT(*) AS actual_rows,
       (SELECT COUNT(DISTINCT unit_nk) FROM lue_dw.dim_unit WHERE unit_key > 0)
     * (SELECT COUNT(*) FROM lue_dw.dim_date
        WHERE date_key > 0 AND full_date <= CURRENT_DATE) AS expected_rows
FROM lue_dw.fact_unit_occupancy_daily;   -- every unit, every day: the two must match
```

7. fact_maintenance_request

Grain (Accumulating snapshot):

> ***One row represents one maintenance request across its whole lifecycle, updated as it is assigned, resolved and verified.***
>
> **Business process:** P3 Maintenance
> **Primary key:** request_sk
> **Load pattern:** Insert when raised; UPDATE milestone date keys and measures as later statuses arrive
> **Expected rows (3 years):** ≈ 6,000
> **This grain answers:** Lags between milestones, SLA breaches, cost by vendor and category, open requests.
> **This grain cannot answer:** Each individual status change after the four milestones (kept in Silver).

Dimensions at this grain

- raised_date_key → dim_date
- assigned_date_key → dim_date
- resolved_date_key → dim_date
- verified_date_key → dim_date
- unit_key → dim_unit
- tenant_key → dim_tenant
- owner_key → dim_owner
- maint_category_key → dim_maintenance_category
- vendor_key → dim_vendor
- request_flags_key → dim_request_flags
- weather_key → dim_weather

Measures and additivity

- **estimated_cost, actual_cost** — Additive. SUM.
- **sla_breached_flag** — Additive count. SUM / COUNT for a rate.
- **days_to_assign, days_to_resolve, days_to_verify** — Non-additive. AVG or median.

Example row

```
request_sk                  5120
raised_date_key             20250714
assigned_date_key           20250714
resolved_date_key           20250716
verified_date_key           20250717
unit_key                    117
tenant_key                  402
owner_key                   57
maint_category_key          12
vendor_key                  31
request_flags_key           3
weather_key                 2911
request_number              MR-2025-02211
request_status              Verified
days_to_assign              0.40
days_to_resolve             2.10
days_to_verify              0.90
estimated_cost              6500.00
actual_cost                 7200.00
sla_breached_flag           0
```

Grain check, run after every load

```
SELECT request_number, COUNT(*)
FROM lue_dw.fact_maintenance_request
GROUP BY request_number
HAVING COUNT(*) > 1;          -- one row per request for its whole life
```

8. fact_lease_event

Grain (Factless (event tracking)):

> ***One row represents one lifecycle event (signed, renewed, notice given or terminated) on one lease.***
>
> **Business process:** P4 Lease lifecycle
> **Primary key:** lease_event_sk
> **Load pattern:** Incremental insert by event date watermark
> **Expected rows (3 years):** ≈ 8,000
> **This grain answers:** Counts of signings, renewals, notices and terminations by building and period.
> **This grain cannot answer:** How long a unit then stayed vacant (needs the occupancy snapshot).

Dimensions at this grain

- event_date_key → dim_date
- unit_key → dim_unit
- tenant_key → dim_tenant
- lease_key → dim_lease
- owner_key → dim_owner
- lease_event_type_key → dim_lease_event_type

Measures and additivity

- **event_count** — Additive count. SUM = COUNT(*).

Example row

```
lease_event_sk              7001
event_date_key              20260215
unit_key                    117
tenant_key                  402
lease_key                   2210
owner_key                   57
lease_event_type_key        2
event_count                 1
```

Grain check, run after every load

```
SELECT lease_key, lease_event_type_key, event_date_key, COUNT(*)
FROM lue_dw.fact_lease_event
GROUP BY lease_key, lease_event_type_key, event_date_key
HAVING COUNT(*) > 1;          -- must return no rows
```

9. fact_vacancy_incentive_coverage

Grain (Factless (coverage)):

> ***One row represents one incentive offer being available on one vacant unit on one calendar day.***

Business process: P5 Vacancy marketing
Primary key: (coverage_date_key, unit_key, incentive_key)
Load pattern: Derived from S6 offer date ranges × dim_date, restricted to vacant unit-days
Expected rows (3 years): Set in Phase 2 (depends on generator settings)
This grain answers: Which offers existed on which vacant units and days, even with no result.
This grain cannot answer: Whether a lease followed, unless combined with lease events or the snapshot.

Dimensions at this grain

- coverage_date_key → dim_date
- unit_key → dim_unit
- incentive_key → dim_incentive
- owner_key → dim_owner

Measures and additivity

- **(none)** — —. COUNT(*) = offer-days.

Example row

```
coverage_date_key           20260110
unit_key                    117
incentive_key               2
owner_key                   57
```

Grain check, run after every load

```
SELECT c.coverage_date_key, c.unit_key
FROM lue_dw.fact_vacancy_incentive_coverage c
JOIN lue_dw.fact_unit_occupancy_daily o
  ON o.snapshot_date_key = c.coverage_date_key AND o.unit_key = c.unit_key
WHERE o.is_occupied = 1;      -- offers exist only on vacant unit-days
```